

Linux 系统 Socket 编程

✦ 一、Socket 编程的基本流程概览

服务器端：

```
socket() → bind() → listen() → accept() → recv()/send() → close()
```

客户端：

```
socket() → connect() → send()/recv() → close()
```

🧱 二、核心函数详解

1 socket()

作用：创建一个“套接字”，即通信端点。

```
int socket(int domain, int type, int protocol);
```

参数：

参数	含义
domain	地址族 (Address Family) 。常见： AF_INET：IPv4 AF_INET6：IPv6 AF_UNIX：本地进程通信
type	套接字类型。常见： SOCK_STREAM：面向连接 (TCP) SOCK_DGRAM：无连接 (UDP)
protocol	通常为 0 (自动选择默认协议)，TCP 对应 IPPROTO_TCP，UDP 对应 IPPROTO_UDP。

返回值：

返回值	含义
>=0	成功，返回“套接字描述符” (类似文件描述符)
-1	失败，错误原因存于 errno

示例：

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
if (sockfd == -1) {
    perror("socket");
    exit(1);
}
```

2 bind()

作用：为套接字绑定本地 IP 地址和端口号。

```
int bind(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

参数：

参数	含义
sockfd	由 socket() 返回的套接字描述符
addr	指向 sockaddr 结构体（通常是 sockaddr_in）的指针
addrlen	结构体的长度（sizeof(struct sockaddr_in)）

结构体：

```
struct sockaddr_in {
    sa_family_t sin_family; // 地址族, AF_INET
    in_port_t sin_port;     // 端口号（必须用 htons() 转换为网络字节序）
    struct in_addr sin_addr; // IP 地址（网络字节序）
};

struct in_addr {
    uint32_t s_addr; // IP 地址
};
```

返回值：

返回值	含义
0	成功
-1	失败（端口被占用等），可用 perror("bind") 输出错误

示例：

```
struct sockaddr_in server_addr;
server_addr.sin_family = AF_INET;
server_addr.sin_port = htons(8080);
server_addr.sin_addr.s_addr = INADDR_ANY; // 绑定所有可用网卡

if (bind(sockfd, (struct sockaddr*)&server_addr, sizeof(server_addr)) == -1) {
    perror("bind");
    exit(1);
}
```

3 listen()

作用：将套接字设为被动模式，准备接受连接（仅 TCP）。

```
int listen(int sockfd, int backlog);
```

参数：

参数	含义
sockfd	已经绑定地址的套接字
backlog	内核允许的最大等待队列长度（通常 5 或更大）

返回值：

返回值	含义
0	成功
-1	失败

示例：

```
if (listen(sockfd, 5) == -1) {
    perror("listen");
    exit(1);
}
```

4 accept()

作用：从等待队列中取出一个连接，建立连接后返回新的套接字。

```
int accept(int sockfd, struct sockaddr *addr, socklen_t *addrlen);
```

参数：

参数	含义
sockfd	服务器监听套接字
addr	用于保存客户端地址信息的结构体指针
addrlen	输入输出参数，传入结构体长度，返回实际长度

返回值：

返回值	含义
>=0	成功，返回一个“已连接套接字”描述符，用于数据传输
-1	失败

示例：

```
struct sockaddr_in client_addr;
socklen_t len = sizeof(client_addr);

int connfd = accept(sockfd, (struct sockaddr*)&client_addr, &len);
if (connfd == -1) {
    perror("accept");
    exit(1);
}

printf("Client connected: %s:%d\n",
       inet_ntoa(client_addr.sin_addr),
       ntohs(client_addr.sin_port));
```

5 connect()

作用：客户端向服务器发起连接请求（仅 TCP）。

```
int connect(int sockfd, const struct sockaddr *addr, socklen_t addrlen);
```

参数：

参数	含义
sockfd	套接字描述符
addr	服务器地址信息
addrlen	地址结构长度

返回值：

返回值	含义
0	成功
-1	失败（连接被拒绝等）

示例：

```
struct sockaddr_in serv_addr;
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &serv_addr.sin_addr);

if (connect(sockfd, (struct sockaddr*)&serv_addr, sizeof(serv_addr)) == -1) {
    perror("connect");
    exit(1);
}
```

6 send() 和 recv()

作用：在已连接的套接字上收发数据（TCP）。

```
ssize_t send(int sockfd, const void *buf, size_t len, int flags);
ssize_t recv(int sockfd, void *buf, size_t len, int flags);
```

参数：

参数	含义
sockfd	已连接的套接字描述符
buf	发送/接收缓冲区
len	缓冲区长度
flags	一般填 0（特殊控制用）

返回值：

返回值	含义
>0	实际发送/接收的字节数
0	对方关闭连接
-1	失败（EAGAIN、EWOULDBLOCK等）

示例:

```
char buffer[1024];
send(connfd, "Hello client!", 13, 0);

int n = recv(connfd, buffer, sizeof(buffer), 0);
buffer[n] = '\0';
printf("Received: %s\n", buffer);
```

7 sendto() 和 recvfrom() (用于 UDP)

```
ssize_t sendto(int sockfd, const void *buf, size_t len, int flags,
               const struct sockaddr *dest_addr, socklen_t addrlen);
ssize_t recvfrom(int sockfd, void *buf, size_t len, int flags,
                 struct sockaddr *src_addr, socklen_t *addrlen);
```

区别:

- UDP 不需要 `connect()` 或 `accept()`。
- 每次发送都要指定目标地址。

示例:

```
sendto(sockfd, "hello", 5, 0, (struct sockaddr*)&client, sizeof(client));

char buf[1024];
socklen_t len = sizeof(client);
recvfrom(sockfd, buf, sizeof(buf), 0, (struct sockaddr*)&client, &len);
```

8 close()

作用: 关闭套接字, 释放资源。

```
int close(int sockfd);
```

返回 0 表示成功, -1 表示失败。

9 地址与字节序辅助函数

函数	作用
<code>htons()</code>	主机字节序 → 网络字节序 (short)
<code>htonl()</code>	主机字节序 → 网络字节序 (long)
<code>ntohs()</code>	网络字节序 → 主机字节序 (short)
<code>ntohl()</code>	网络字节序 → 主机字节序 (long)

10 IP 地址转换函数

函数	作用
<code>inet_addr("127.0.0.1")</code>	IP 字符串 → 网络字节序整数
<code>inet_ntoa(in_addr)</code>	网络字节序整数 → 字符串
<code>inet_pton(AF_INET, "127.0.0.1", &addr)</code>	字符串 → 二进制
<code>inet_ntop(AF_INET, &addr, buf, sizeof(buf))</code>	二进制 → 字符串

✓ 三、TCP 简单示例

Server:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in addr;
addr.sin_family = AF_INET;
addr.sin_port = htons(8080);
addr.sin_addr.s_addr = INADDR_ANY;

bind(sockfd, (struct sockaddr*)&addr, sizeof(addr));
listen(sockfd, 5);

struct sockaddr_in client;
socklen_t len = sizeof(client);
int connfd = accept(sockfd, (struct sockaddr*)&client, &len);

char buf[100];
recv(connfd, buf, sizeof(buf), 0);
printf("From client: %s\n", buf);
send(connfd, "Hello!", 6, 0);

close(connfd);
close(sockfd);
```

Client:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
struct sockaddr_in serv;
serv.sin_family = AF_INET;
serv.sin_port = htons(8080);
inet_pton(AF_INET, "127.0.0.1", &serv.sin_addr);

connect(sockfd, (struct sockaddr*)&serv, sizeof(serv));
send(sockfd, "Hi server!", 10, 0);

char buf[100];
recv(sockfd, buf, sizeof(buf), 0);
printf("From server: %s\n", buf);
```

```
close(sockfd);
```

四、补充：常见错误与调试技巧

错误提示	原因
<code>bind: Address already in use</code>	端口未释放，可加 <code>setsockopt(... SO_REUSEADDR ...)</code>
<code>connect: Connection refused</code>	服务器未启动或端口错误
<code>recv() = 0</code>	对方关闭连接
<code>send() < len</code>	TCP 可能分段发送，应循环发送